What Are We Measuring?

An Instrument and Implementation Audit of Energy Comparisons Across Deep Learning Ecosystems

Leonardo Pampaloni Marco Pagliocca Enrico Vicario
Roberto Verdecchia
University of Florence, Italy

August 19, 2026

Abstract

Comparing the energy cost of deep learning across language–framework ecosystems is an attractive idea: the practitioner picks a stack, and the choice appears to carry an energy price. We show that such comparisons are dominated by decisions that have nothing to do with the ecosystems being compared.

We audit a complete measurement campaign of 2880 tracked blocks — two CNN architectures, three image datasets, eight ecosystems, on one NVIDIA L40S server — and re-analyse it end to end. Three classes of defect emerge, each quantified. *Instrument*: the campaign mixed two CodeCarbon major versions, two tracking modes and two sampling intervals; for four of the eight stacks roughly two thirds of the reported energy is a constant, modelled host power multiplied by wall-clock time. *Implementation*: the stacks did not run the same experiment — two learning rates, four data-loader configurations, one stack normalising its inputs, one training on $28 \times 28 \times 1$ inputs while evaluating on $32 \times 32 \times 3$, and one evaluating a single image at a time. *Placement and linkage*: a framework whose CUDA libraries fail to load, or a binary whose linker drops the CUDA dependency, runs the entire workload on the CPU while reporting nothing but a warning.

Under a common measurement boundary the study's headline conclusions do not survive. The reported "faster is not greener" effect disappears: the energy and time rankings agree with Spearman $\rho = 1.00$ for inference and 0.98 for training. The four stacks that share the LibTorch backend — the study's own internal control group — span 98 % to 100 % of the total reported spread, so the effect is host-side rather than language-intrinsic.

We contribute a written experiment specification, a shared measurement bridge and run contract used by all seven in-scope ecosystems, and an automated conformance checker (56 checks). With these in place, five independent implementations of ResNet-18 reach test accuracies within 3.6 % of one another on CIFAR-100 after one epoch — a check the original campaign could not perform, because no ecosystem wrote its accuracy to disk.

1 Introduction

The energy cost of deep learning is now a routine object of study, and a natural question follows: does the choice of programming language and framework carry an energy price? The question is well posed. A practitioner selecting between PyTorch, LibTorch, DL4J or an R binding is choosing a whole stack — a binding layer, a data pipeline, a runtime, a toolchain — and it is plausible that the choice shows up in the electricity bill.

This paper is about what it takes to answer that question, and what happens when the necessary care is not taken. We take as our object of study a complete campaign: ResNet-18 and VGG-16, on Fashion-MNIST, CIFAR-100 and Tiny ImageNet, across eight language–framework ecosystems, measured with CodeCarbon on a dedicated NVIDIA L40S server, 2880 tracked measurement blocks in total. The campaign was carefully executed by its authors and

reports a coherent story: compiled stacks are more efficient, inference and training rank differently, and execution time is an unreliable proxy for energy.

We re-analyse that campaign from its raw logs and audit the source of all eight implementations. We find that the reported differences are largely produced by the measurement apparatus and by implementation drift between the stacks, and that the headline conclusions do not survive a common measurement boundary. None of the defects we describe is exotic; several are invisible in the data and detectable only by reading the code that produced it.

Contributions.

1. A quantified audit of a real campaign, separating what the data can support from what the instrument produced (Sections 3 and 4).
2. A re-analysis showing that the original conclusions do not hold under a common measurement boundary (Section 5).
3. A catalogue of defect classes for energy studies of this shape, each with a measured effect size (Section 6).
4. A specification, a shared measurement contract implemented by seven ecosystems, and an automated conformance checker (Section 7).

Scope and honesty about it. This paper does not report a corrected ranking of ecosystems. The corrected measurement infrastructure exists and every stack has been verified to run on the accelerator and to record its accuracy, but the replicated campaign that would produce such a ranking is reported separately (Section 8). We consider a ranking produced before the defects below are fixed to be worse than no ranking at all, and the point of this paper is to say why.

2 Background: the campaign under study

The campaign covers eight language–framework ecosystems: Python with PyTorch, TensorFlow and JAX; C++ with LibTorch; Java with Deeplearning4j; R with `torch`; Rust with `tch`; and MATLAB with the Deep Learning Toolbox. Two architectures (ResNet-18, VGG-16) are trained on three datasets (Fashion-MNIST, CIFAR-100, Tiny ImageNet), all resized to 32×32, for 30 epochs with Adam, batch size 128, on a single NVIDIA L40S.

Energy is measured with CodeCarbon, one tracker per epoch per phase, giving 2880 tracked blocks: 1440 training and 1440 inference. Each configuration was executed exactly once; the 30 epochs of a run were treated as repeated measurements.

A unit error, and why it matters here. CodeCarbon reports energy in kilowatt-hours. The campaign's tables report the same numbers labelled as Joules — an offset of 3.6×10^6. The error is verifiable three ways from the data alone: the ratio of the emissions column to the energy column is 0.3306, which read as kW h gives $331\,\mathrm{g\,kW^{-1}\,h}$, the grid intensity CodeCarbon used; the implied mean power read as kW h is 120 W to 511 W, plausible for this hardware, while read as Joules it is 3×10^{-5} W; and the columns are additive, so all four share one unit. The corrected campaign totals are 4.02 kW h and 1.33 kg CO_2eq.

We mention this first not because it is the most serious problem — relative rankings are unaffected — but because it is the only one visible without reading the code, and it is the one a reader is most likely to catch.

Table 1: Instrument configuration per ecosystem, read from the source of each bridge. Two CodeCarbon major versions, two tracking modes and two sampling intervals were in use simultaneously.

Ecosystem	CodeCarbon	Tracking mode	`measure_power_secs`	Host share
Python/PyTorch	2.8.4	machine	15 (default)	63 %
Python/TensorFlow	2.8.4	machine	15 (default)	69 %
Python/JAX	2.8.4	machine	1	63 %
Rust/tch	2.8.4	machine	1	63 %
R/torch	2.8.4	**process**	1	31 %
C++/LibTorch	3.0.4	machine	15 (default)	42 %
Java/DL4J	3.0.4	machine	15 (default)	40 %
MATLAB/DLT	3.0.4	machine	15 (default)	55 %

3 The instrument was not the same across ecosystems

Each non-Python ecosystem drives CodeCarbon through its own Python bridge. The campaign shipped five such bridges, and they were configured four different ways. Table 1 reports what each stack actually used, read from source.

3.1 Two thirds of the reported energy is a function of wall-clock time

The difference between CodeCarbon 2.8.x and 3.0.x is not cosmetic. Where RAPL is unavailable, 2.8.x substitutes a hardcoded constant for CPU power ($85\,\mathrm{W} \times 0.5 = 42.5\,\mathrm{W}$), and models RAM power as $0.375\,\mathrm{W\,GiB^{-1}}$ of *installed* memory — on this server, 503 GiB, giving a constant 188.84 W regardless of use.

For the four stacks on 2.8.4 in machine mode, the identity

$$\mathrm{cpu\_energy} + \mathrm{ram\_energy} = 231.3\,\mathrm{W} \times \mathrm{duration}$$

holds to within 0.5 %. Roughly two thirds of their reported energy is therefore a deterministic linear function of wall-clock time, computed from a constant that depends on how much memory is physically installed in the machine. The 3.0.x stacks carry about 107 W of modelled host power instead, and R — the only stack in process mode — carries almost none: its RAM energy share is 0.03 % against roughly 50 % elsewhere.

Comparing raw `energy_consumed` across these stacks therefore compares instrument configurations at least as much as it compares ecosystems.

3.2 The CodeCarbon version was an ambient property of the shell

The Rust bridge launched its measurement daemon with `Command::new("python3")` — the first interpreter on `PATH`. Which CodeCarbon version measured that ecosystem was thus a property of the shell that happened to start the run, not a decision of the experiment. This is the mechanism by which one campaign came to mix CodeCarbon 2.8.4 and 3.0.4.

3.3 The bridge was not synchronised with the workload

Of the three daemon-style bridges, one wrote `START` into the daemon's stdin and began computing immediately, without waiting for the tracker to exist. CodeCarbon takes seconds to initialise, so the tracked window did not cover the work it was meant to measure. Measured on an RTX 3090 for one epoch of ResNet-18/CIFAR-100:

Block	Unsynchronised	Synchronised
Inference	0 J (on real GPU work)	97 J
Training	861 J	1127 J

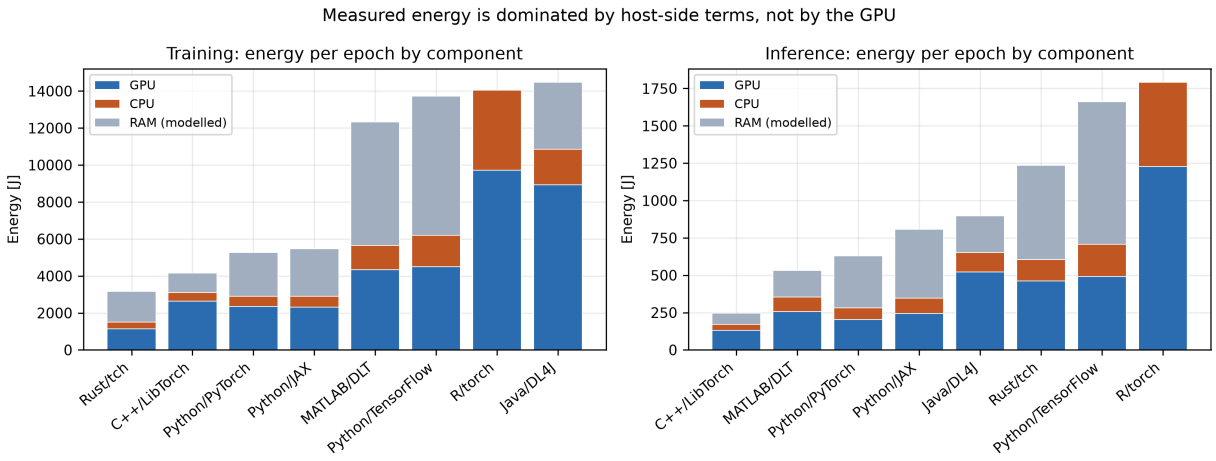

Figure 1: Energy per epoch split into GPU, CPU and modelled RAM. The RAM term is not measured: for the CodeCarbon 2.8.x stacks it is a constant 188.84 W derived from *installed* memory, so it grows only with wall-clock time. It is the largest single component for four of the eight ecosystems.

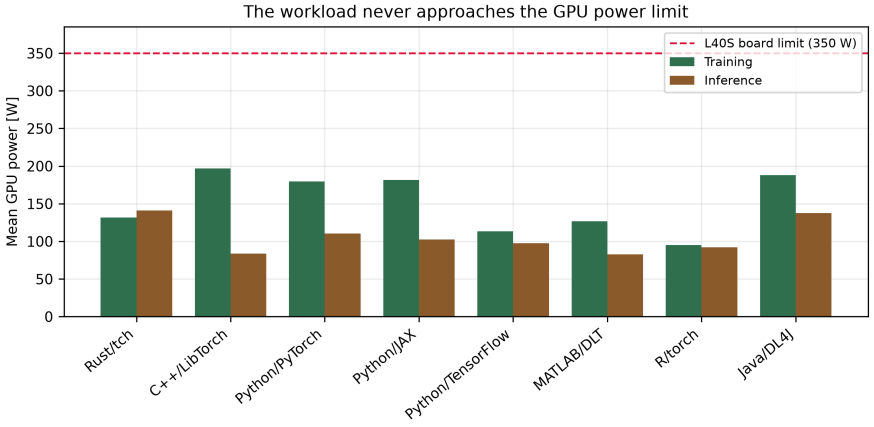

Figure 2: Mean GPU power per ecosystem and phase, against the board limit of the measurement device. No configuration approaches the limit in either phase.

The inference block recorded nothing at all; the training block was underestimated by 24 %. The other two bridges were synchronous: the Java bridge waits on a ready latch, and the C++ bridge embeds the interpreter and is synchronous by construction. This defect is therefore not a property of the approach but of one implementation of it — which is precisely why it survived.

3.4 The workload does not load the accelerator

Mean GPU power derived from the energy counter ranges from 83 W to 197 W against a 350 W board limit, and never approaches the limit in any configuration. Between 30 % and 69 % of the measured energy is GPU energy; the remainder is host-side. At that utilisation the campaign measures data loading and dispatch more than it measures deep learning (Figure 2).

4 The ecosystems did not run the same experiment

The manuscript states a common protocol: the same 30 epochs, learning rate, optimiser and batch size. Reading the source of all eight implementations shows otherwise.

Table 2: Training protocol as implemented, read from source. The manuscript states a common learning rate; two values are in use.

Ecosystem	Optimiser	LR	Loader mechanism	Threads	Input scaling
Python/PyTorch	Adam	1e-4	DataLoader workers	2	$[0, 1]$
Python/TensorFlow	Adam	**1e-3**	Keras generator	1	$[0, 1]$
Python/JAX	Adam	1e-4	Keras generator	1	$[0, 1]$
C++/LibTorch	Adam	1e-4	DataLoader workers	2	$[0, 1]$
Java/DL4J	Adam	1e-4	AsyncDataSetIterator	2	$[0, 1]$
R/torch	Adam	1e-4	dataloader workers	**0**	$[0, 1]$
Rust/tch	Adam	**1e-3**	rayon, all cores	**96**	**mean/std**
MATLAB/DLT	adam	1e-4	augmentedImageDatastore	1	$[0, 1]$

4.1 Data-loader parallelism, not language, tracks the ranking

The largest single difference is how many CPU threads decode and feed images: from zero (R decodes on the main thread) to all 96 cores (Rust, through `rayon`). Spearman correlation between loader threads and mean epoch duration is $\rho = -0.73$ ($p = 0.04$) across the eight ecosystems. The fastest stack decodes on 96 cores; the slowest uses no loader workers; they differ by 11.6× in duration.

Combined with the GPU-load result of Section 3 — the accelerator at a fraction of its limit, duration accounting for essentially the whole log energy spread — the most parsimonious reading of the headline result is that it ranks data-pipeline configurations.

4.2 One ecosystem was not solving the same task

Three defects in the Rust implementation all push in the direction of the reported result:

- **Fashion-MNIST was trained at** $28 \times 28 \times 1$**.** The loader applied no resize and no channel replication, so training ran on the native grayscale image while *its own evaluation loop* — and every other ecosystem, in both phases — used $32 \times 32 \times 3$. The `conv1` input volume during training was 0.26× everyone else's.
- **Tiny ImageNet was trained at its native** 64×64 in the ResNet binary, while the VGG binary passed `Some(32)` and every other ecosystem used 32×32: four times the spatial work for the same nominal configuration, inside one ecosystem.
- **Evaluation ran one image at a time** (`iter_batches(1)`) in all six binaries, against batch 128 elsewhere. Batch-1 GPU inference is launch-overhead bound.

The published numbers line up with the defects. Against the median of the other stacks, this ecosystem's training energy is 0.07× for ResNet-18/Fashion-MNIST — the cheapest cell in the entire campaign, and precisely the cell with the smallest input — rising to 0.61×; its inference energy is 1.51–1.95× on CIFAR-100 and Fashion-MNIST.

The manuscript's *training/inference ranking reversal*, reported as a finding, is therefore reproducible for this ecosystem as a pure artefact of input shape and evaluation batching.

4.3 One ecosystem evaluated in training mode

The TensorFlow stack built its functional graph with `backbone(inputs, training=True)`, pinning training-mode behaviour into the graph. Batch normalisation then used the statistics of the current batch during evaluation as well. Because the test split is served in class order, each evaluation batch held a single class and normalisation ran against degenerate per-class statistics: measured test accuracy was 21 % against 83 % on train, and 86 % once the flag was removed.

Table 3: Best and worst ecosystem, and spread, under each energy definition. The identity of the worst ecosystem in training changes with the definition.

Phase	Energy definition	Best / Worst	Spread
Training	as measured	Rust/tch / Java/DL4J	4.58×
Training	GPU only	Rust/tch / R/torch	8.54×
Training	harmonised	Rust/tch / R/torch	9.94×
Training	execution time	Rust/tch / R/torch	11.63×
Inference	as measured	C++/LibTorch / R/torch	7.28×
Inference	GPU only	C++/LibTorch / R/torch	9.30×
Inference	harmonised	C++/LibTorch / R/torch	8.83×
Inference	execution time	C++/LibTorch / R/torch	8.47×

Two consequences. The accuracy this stack would have reported was meaningless — but accuracy was never written to disk, so nothing surfaced it. And its inference phase was measuring a different computation from every other ecosystem, all of which switch to evaluation mode.

4.4 The dataset itself was ambiguous

Fashion-MNIST class 0 is `T-shirt/top`. Used verbatim as a directory name it creates a *nested* directory, so `train/T-shirt/` holds a `top/` subdirectory rather than images. Loaders then disagree about what the dataset contains: `ImageFolder` and `flow_from_directory` walk recursively and recover the 6000 images, a loader listing only direct children sees an empty class, and a loader using `read_dir` yields the subdirectory as an image path. The same dataset directory is three different datasets depending on who reads it.

5 Re-analysis under a common measurement boundary

Because the instrument differed across stacks, we report every result under three energy definitions:

as measured CodeCarbon's total, instrument-confounded;

GPU only the NVML energy counter alone — the same instrument everywhere, and the only quantity attributable to the deep learning computation;

harmonised GPU counter plus a single uniform 107 W host model applied to every ecosystem.

5.1 The rankings are not robust to the boundary

The headline figures quoted in the campaign — 4.6× for training, 7.3× for inference — are the as-measured spreads. Under a common boundary they become 9.9× and 8.8×, and the worst ecosystem in training changes identity.

5.2 "Faster is not greener" does not survive

The campaign concludes that execution time is an unreliable proxy for energy. We quantify the relationship rather than reading it off qualitative quadrants.

Within an ecosystem, energy is very nearly a deterministic linear function of wall-clock time: the median R^2 of energy on duration is 0.98. This is expected once Section 3 is taken into account.

Across ecosystems, agreement between the energy ranking and the time ranking is:

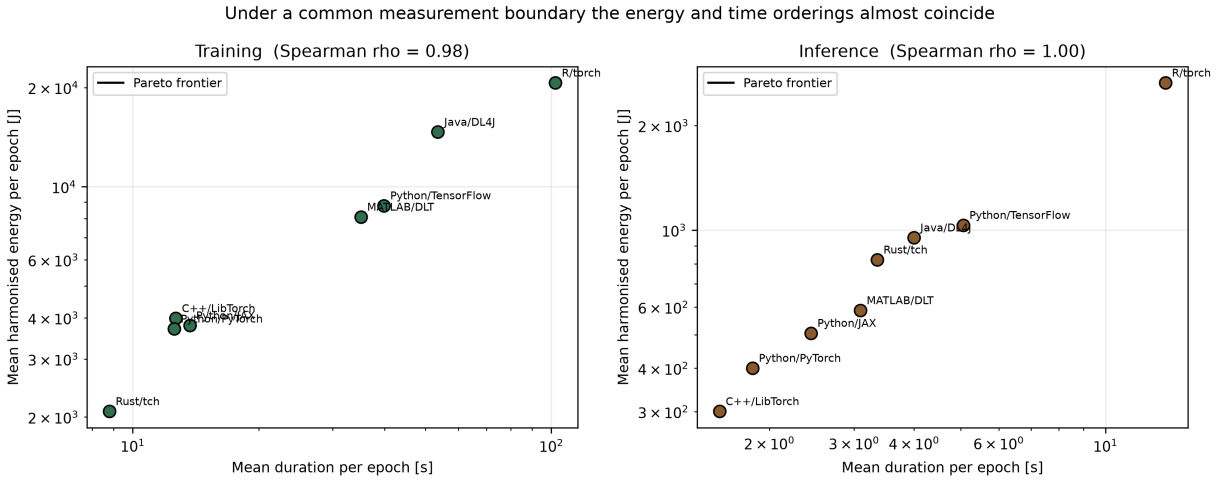

Figure 3: Energy against duration per ecosystem under the harmonised boundary. The orderings coincide for inference and differ by one pair in training; the campaign's qualitative quadrant plot suggested a much weaker relationship.

Phase	Energy definition	Spearman ρ	Discordant pairs
Inference	as measured	0.90	3 of 28
Inference	GPU only	0.98	1 of 28
Inference	harmonised	**1.00**	**0 of 28**
Training	as measured	0.95	2 of 28
Training	GPU only	0.93	2 of 28
Training	harmonised	**0.98**	**1 of 28**

Under a harmonised boundary the disagreement vanishes entirely for inference and reduces to a single pair in training. The specific examples the manuscript gives are exactly the pairs that disappear once the host power model is held constant.

Decomposing energy as mean power times duration, 98 % (inference) and 107 % (training) of the log spread is attributable to duration, with a mean-power spread of only 1.3–1.6×.

5.3 The internal control group reproduces the whole effect

Four of the eight ecosystems — Python/PyTorch, C++/LibTorch, R/torch and Rust/tch — run the same LibTorch kernels. They are the study's natural control group: any difference between them is binding, runtime and data-pipeline overhead, not language.

Restricting the comparison to those four reproduces 98 % to 100 % of the full eight-stack spread on a log scale, in every phase and under every energy definition. Whatever is being measured is host-side.

We note that the group was not in fact controlled: the four stacks ran LibTorch 2.1.0, 2.5.x, 2.6.0 and 2.7.0 respectively — six minor releases apart — so any difference between them also mixes in kernel, allocator and cuDNN changes.

5.4 What the design can and cannot support

Each of the 48 configurations was executed once, and the 30 epochs of a run are repeated measurements of that run, sharing one initialisation, one JIT outcome, one allocator state and one thermal trajectory. The effective number of independent observations per configuration is one, not thirty.

We therefore report medians alongside means, bootstrap intervals over epochs labelled explicitly as within-run dispersion, and effect sizes; and we do not report a significance claim about

Table 4: Defect classes, with the measured effect of each. "Visible in the data" asks whether a reader given only the released measurements could detect the problem.

Class	Measured effect	Visible
Unit mislabelling	Energy reported in kW h labelled as J; 3.6×10^6	yes
Mixed instrument versions	231 W vs 107 W of modelled host power; up to 70 % of reported energy	no
Mixed tracking mode	One stack at 31 % host share against ∼50 %	no
Interpreter resolved from `PATH`	CodeCarbon version becomes a property of the shell	no
Unsynchronised tracker	Inference 0 J recorded; training underestimated 24 %	no
Divergent hyperparameters	Two learning rates across eight stacks	no
Divergent loader parallelism	0 to 96 threads; $\rho = -0.73$ with duration, 11.6× spread	no
Divergent input shape	One stack at 0.26× the `conv1` input volume; 0.07× energy	no
Divergent evaluation batching	Batch 1 vs 128; 1.5–1.95× inference energy	no
Evaluation in training mode	Test accuracy 21 % vs 86 %; different computation measured	no[†]
Silent CPU fallback	Whole workload on CPU after a CUDA wheel mismatch; one warning line	no
Linker drops the CUDA dependency	Same source, same LibTorch: `Cpu` vs `Cuda(0)`	no
Ambiguous dataset layout	A class name containing `/` yields three different datasets	no
Host not idle	Whole-machine tracking counts unrelated downloads and builds	no

[†] invisible because no ecosystem wrote its accuracy to disk; visible the moment one does.

ecosystems. Tests computed over epochs are anti-conservative by an unknown factor: they compare the epochs of one run with the epochs of another.

Separately, 158 rows (5.5 %) report a sampled GPU power of 0 W and 27 exceed the board limit, up to 3054 W. These are confined to CodeCarbon's *sampled* power columns, which are unreliable because 69 % of the measured blocks are shorter than the 15 s default sampling period. The energy columns come from the NVML counter and show no such values.

6 A catalogue of defect classes

Table 4 collects the defects we found, each with a measured effect. We report them as classes rather than as a list of bugs in one project: every one of them is a plausible mistake in any study of this shape, and most are invisible in the resulting data.

Two observations. First, only one of the fourteen is visible from the released data — the one that changes no conclusion. Second, several are not mistakes of carelessness but defaults: the linker's `-as-needed` behaviour, CodeCarbon's fallback power constants, a framework resolving CUDA wheels that do not match its build. A study can be executed carefully and still land on

all of them.

This is our argument for enforcement rather than discipline. The defects that matter are the ones that produce plausible numbers, and plausible numbers are exactly what review cannot catch.

7 Making the comparison possible

The defects in Sections 3 and 4 share a shape: each is a place where seven independent implementations were free to differ, and did. Our response is to remove the freedom where it is not part of the research question, and to make the remaining differences fail loudly.

7.1 A written specification

We state the experiment once, as a document rather than as folklore: model source, optimisation, data pipeline, backend, measurement and replication. Anything the specification does not pin is a documented degree of freedom, and anything it pins is checkable.

7.2 One measurement bridge and one run contract

The five per-stack CodeCarbon bridges are replaced by one. Every ecosystem drives it through a line protocol (`START`, `STOP`, `METRIC`, `EXIT`), all of which are synchronous, and reads its parameters from a shared environment contract: output directory, ecosystem, model, dataset, repetition, seed, epoch count, data and model roots, and the interpreter to measure with.

Environment variables rather than command-line flags: adding argument parsing to a C++ binary, a Maven target, an `Rscript` and a Rust binary is four different pieces of plumbing, and four places for the stacks to drift apart again.

Two things the contract makes impossible rather than merely discouraged. The interpreter is named explicitly, so no stack can silently be measured by whatever `python3` the shell resolves. And the harness refuses to start when the framework cannot see the accelerator, so a CPU fallback aborts the run instead of being recorded as an ecosystem property.

7.3 One model where the backend allows it

For the stacks that share LibTorch, the model is exported once as TorchScript from `torchvision` and loaded by each binding. Verified end to end: all six modules load into Rust and take an optimiser step through the variable store, with parameter counts matching the Python export exactly (11,227,812 for ResNet-18/CIFAR-100), and all six load and forward in R.

R is a documented exception. In `torch` 0.17.0 a script module's `$train()` and `$eval()` raise `unused argument`, and the underlying handle is not reachable through the object's documented fields, so a loaded module cannot be switched between training and evaluation. Since the shared modules are exported in training mode, this stack would evaluate with batch normalisation using batch statistics — exactly the TensorFlow defect of Section 4. It therefore builds its own model, and the architecture parity that holds for Python, C++ and Rust does not hold for R. That is a property of the binding, and it belongs in the threats to validity rather than being papered over.

7.4 An automated conformance checker

The specification is enforced by a checker that reads the source the campaign actually runs: 56 checks covering learning rate, batch size, loader threads, input scaling, evaluation batching, shared-module use, backend version alignment, bridge use, tracker synchronisation, device assertion and the absence of hard-coded paths.

The checker is the piece we would keep if we could keep only one. It turns "the stacks should be configured the same" from an intention into a claim that fails when it stops being true.

Table 5: Test accuracy after one epoch on CIFAR-100, identical settings and seed. The first campaign could not perform this check: no ecosystem wrote its accuracy to disk.

Ecosystem	Test accuracy	Train loss
Python/PyTorch	19.84 %	3.828
C++/LibTorch	20.01 %	3.822
Rust/tch	20.62 %	3.798
R/torch	20.79 %	3.687
Java/DL4J	17.14 %	3.784

7.5 Every ecosystem verified on the accelerator

All seven in-scope ecosystems now run on the GPU under the shared contract and write per-epoch accuracy. Getting there required per-stack fixes that no documentation mentions: the linker must be told to keep `torch_cuda` or the Rust binary silently runs on the CPU; `CUDA::nvToolsExt` must be defined before `find_package(Torch)` because NVTX left the CUDA toolkit in CUDA 12; the C++ embedded interpreter resolves `site-packages` from whichever libpython it was linked against rather than from the campaign environment; R's `liblantern.so` links against `libcudart.so.12` while the R bundle ships it under a hashed name; and DL4J 1.0.0-M2.1 links against the CUDA 11.6 runtime and is the last release of that API, so on a CUDA 12 host the backend fails with `libcudart.so.11.0: cannot open shared object file` and ND4J reports only "no backend on your classpath".

Table 5 is the check that matters. Five independent implementations agree within 3.6 %, and the four sharing the LibTorch backend agree within one point. Java sits lower, which is consistent with it building its own graph rather than loading the shared module — and that is now something a reader can see rather than something nobody recorded.

8 Threats to validity and what remains

We do not report a corrected ranking. The corrected infrastructure exists and every ecosystem has been verified on the accelerator, but the replicated campaign has not been executed at the time of writing. We consider this the honest position: a ranking produced before the defects of Sections 3 and 4 are fixed would inherit them, and a ranking produced after them is a different experiment that deserves its own reporting.

Residual confounds we could not remove. Two survive by construction rather than by oversight. DL4J 1.0.0-M2.1 links against the CUDA 11.6 runtime and is the last release of its API, so the Java ecosystem cannot be brought onto the CUDA 12 toolchain the other six use; the stack now carries its own redistributables, which makes it reproducible but does not remove the version difference. And the R binding cannot switch a script module between training and evaluation, so it cannot share the traced model. Both are properties of the ecosystems, and arguably part of what one is measuring when one measures an ecosystem — but they are not the binding overhead the study set out to isolate.

The workload. The campaign runs the accelerator at 24 % to 56 % of its power limit, so it measures data loading and dispatch more than deep learning. A replicated campaign should add at least one configuration that saturates the accelerator, and report utilisation alongside energy so a reader can see which regime a result belongs to.

The instrument itself. We now read hardware counters directly — `nvmlDeviceGetTotalEnergyConsumption` for the accelerator and Intel RAPL for the CPU package — and run CodeCarbon over the same

window as a cross-check.

It is worth being precise about what was wrong with the original instrument, because it was not accuracy. Measured on the replication machine with both instruments bracketing the same window, CodeCarbon 3.3.0 agrees with the counters to within 1 % for block durations from 2 s to 40 s, for GPU and CPU alike. The failure mode is that it *substitutes a model when a counter is unavailable, and says so only in a log line*: where RAPL cannot be read the CPU term becomes a constant, the RAM term is derived from installed memory rather than from use, and the model changed between major versions. An instrument that refused to run would have been preferable to one that quietly estimated.

This matters for how a number should be reported. NVML plus RAPL is a *chip-level* boundary: it excludes the power supply, fans and drives that a wall meter would see. Published validations place software estimators within tens of percent of a wall meter, so a chip-boundary figure must not be presented as a whole-system one. The figure the campaign under audit reports sits between the two boundaries and corresponds to neither — for four ecosystems roughly two thirds of it was a model of RAM power derived from how much memory the machine happened to have installed.

We had no wall meter, and say so rather than implying a boundary we did not measure.

Generalisation. Everything here concerns one campaign, one hardware platform and one model class. The defect *classes* of Section 6 are, we believe, general to studies of this shape; the effect sizes are not.

Our own errors. Two of the defects in Section 6 we introduced ourselves while repairing the study, and caught only because the accuracy check of Table 5 existed: switching the Rust stack to the shared TorchScript module made it evaluate in training mode, costing 15 accuracy points, because `TrainableCModule` ignores the training flag passed to `forward_t`. We measured contaminated blocks ourselves by running package downloads while the campaign was tracking whole-machine energy. We report this because it is the argument of the paper in miniature: the defects that matter are the ones that produce plausible numbers, and the only reliable defence is a check that fails.

9 Related work

Cross-language energy comparisons and cross-framework deep learning comparisons are both established. Georgiou et al. compare PyTorch and TensorFlow with statistical tests, effect sizes, comparable accuracy across frameworks and function-level profiling, and report that the energy ranking of deep learning frameworks reverses between training and inference. Alizadeh and Castor measure the energy and inference time of deep learning runtime infrastructures across frameworks and execution providers, and report that efficiency is hard to predict and is driven by GPU and CPU usage. Cross-language studies at the level of general-purpose benchmarks likewise report that faster execution does not imply lower energy.

Our contribution is orthogonal to these. We do not add another ranking; we ask what has to be true of an experimental apparatus before any such ranking means anything, and we show — on a real campaign, with measured effect sizes — how far an apparatus can drift while producing entirely plausible numbers. The phase-reversal result is a useful illustration: we reproduce it for one ecosystem as a pure artefact of evaluation batching and input shape (Section 4).

10 Conclusion

The question of whether language–framework ecosystems differ in energy cost remains open, and we think it is worth answering. What we can say is that the apparatus needed to answer it is

more demanding than it appears.

In the campaign we audited, the instrument was configured five different ways, the workload differed between stacks in learning rate, input shape, evaluation batching and loader parallelism, and two stacks could have been running on the CPU without any recorded evidence either way. Each of these produces plausible numbers. Together they produce a plausible story — one that does not survive a common measurement boundary.

The practical lesson is that a cross-ecosystem energy study needs a written specification and a mechanism that enforces it, in the same way that a performance benchmark needs a warm-up policy. We provide both, and the checker is the part we would keep if we could keep only one thing: it is what turns "the stacks should be configured the same" into a claim that fails loudly when it stops being true.